

Lab Assignment # 06

Course Title : **AI Assistant Coding**
Name of Student : **BG.Sreevani**
Enrollment No. : **2303A54066**
Batch No. : **48**

Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals

Task Description #1: Classes (Student Class)

Task: Use an AI tool to generate a Python program for student information management module.

Prompt: "Generate a Python code Student class with constructor, display_details() method, sample object, and a short code analysis."

Expected Output:

- A Python class with a constructor (`__init__`) and a `display_details()` method.
- Sample object creation and output displayed on the console.
- Brief analysis of AI-generated code.

Output Screenshot:

```
.py x design.py
1 #Generate a Python Student class with attributes: name, roll_number, branch.
2 # Include a method display_details() that prints student information.
3 # Create a sample object and call display_details() to verify output
4 class Student:
5     def __init__(self, name, roll_number, branch):
6         self.name = name
7         self.roll_number = roll_number
8         self.branch = branch
9
10    def display_details(self):
11        print(f"Name: {self.name}")
12        print(f"Roll Number: {self.roll_number}")
13        print(f"Branch: {self.branch}")
14
15    # Create sample objects and call display_details()
16    student1 = Student("Alice", "101", "Computer Science")
17    student1.display_details()
18
19    student2 = Student("Bob", "102", "Mechanical Engineering")
20    student2.display_details()
21
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "C:/Users/Sreevani B G/OneDrive/Desktop/p/AI/.py"
Name: Alice
Roll Number: 101
Branch: Computer Science
Name: Bob
Roll Number: 102
Branch: Mechanical Engineering
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI>
```

Ln 21, Col 1 Spaces: 4 UTF-8 CRLF Python 3.13.9 (Microsoft Store) Go Live

Task Description #2: Loops (Multiples of a Number)

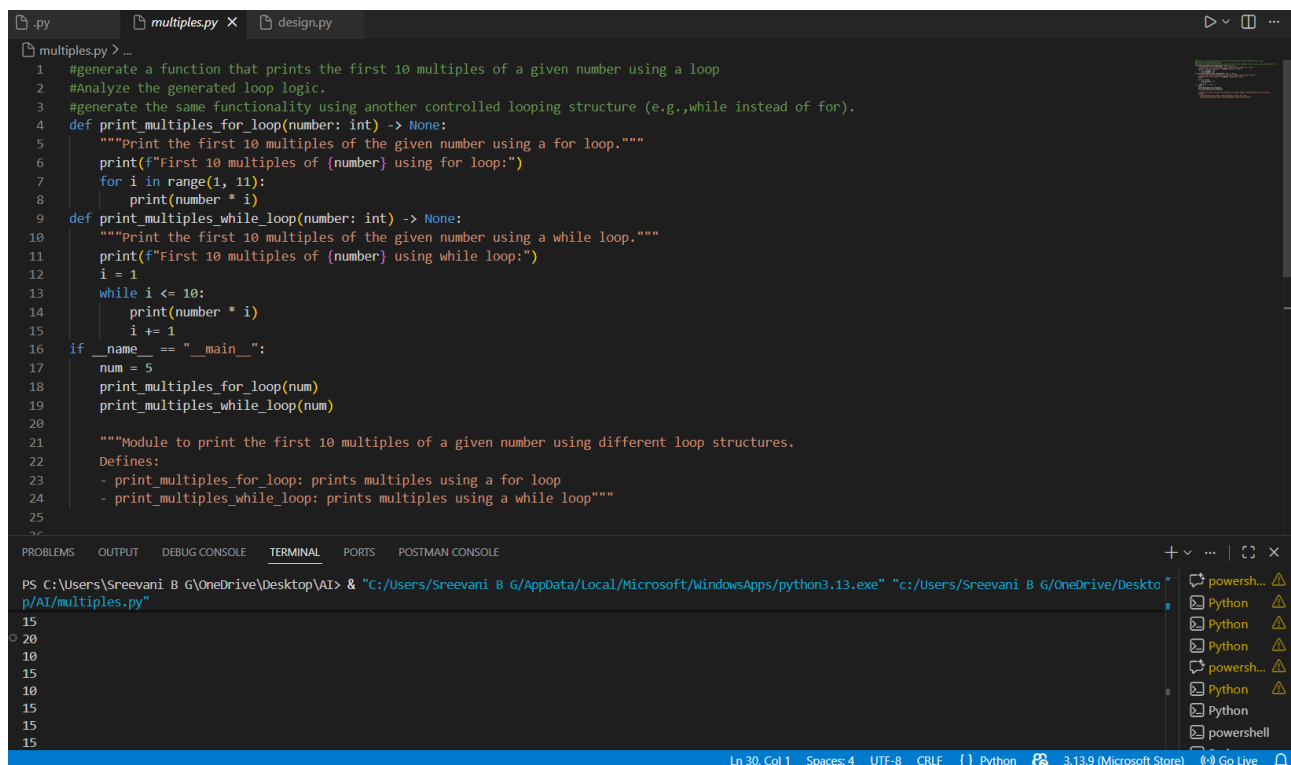
Task: Prompt the AI tool to generate a function that prints the first 10 multiples of a given number using a loop.

prompt: Write and analyze Python code to print the first 10 multiples of a number using both for and while loops

Expected Output:

- Correct loop-based Python implementation.
- Output showing the first 10 multiples of a number.
- Comparison and analysis of different looping approaches.

Output Screenshot:



```
.py  multiples.py  design.py
multiples.py > ...
1 #generate a function that prints the first 10 multiples of a given number using a loop
2 #Analyze the generated loop logic.
3 #generate the same functionality using another controlled looping structure (e.g.,while instead of for).
4 def print_multiples_for_loop(number: int) -> None:
5     """Print the first 10 multiples of the given number using a for loop."""
6     print(f"First 10 multiples of {number} using for loop:")
7     for i in range(1, 11):
8         print(number * i)
9 def print_multiples_while_loop(number: int) -> None:
10    """Print the first 10 multiples of the given number using a while loop."""
11    print(f"First 10 multiples of {number} using while loop:")
12    i = 1
13    while i <= 10:
14        print(number * i)
15        i += 1
16 if __name__ == "__main__":
17     num = 5
18     print_multiples_for_loop(num)
19     print_multiples_while_loop(num)
20
21     """Module to print the first 10 multiples of a given number using different loop structures.
22     Defines:
23     - print_multiples_for_loop: prints multiples using a for loop
24     - print_multiples_while_loop: prints multiples using a while loop"""
25
26
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/Desktop/AI/multiples.py"
15
20
10
15
10
15
15
15
Ln 30, Col 1  Spaces: 4  UTF-8  CRLF  () Python  3.13.9 (Microsoft Store)  Go Live
```

Analysis of for Loop Logic

- The for loop uses range(1, 11) to ensure the loop runs exactly 10 times.
- In each iteration, the loop variable *i* is multiplied with the given number *n* to produce the next multiple.
- The logic is simple and easy to follow, which improves code readability.
- This approach is ideal when the number of iterations is fixed and known beforehand.

Analysis of while Loop Logic

- The while loop keeps running as long as the condition $i \leq 10$ remains true
- The counter variable *i* must be initialized before the loop and updated manually inside the loop.
- This loop provides greater control over how and when the loop executes.
- It can be slightly error-prone if the condition or increment statement is not handled correctly, which may lead to infinite loops.

Task Description #3: Conditional Statements (Age Classification)

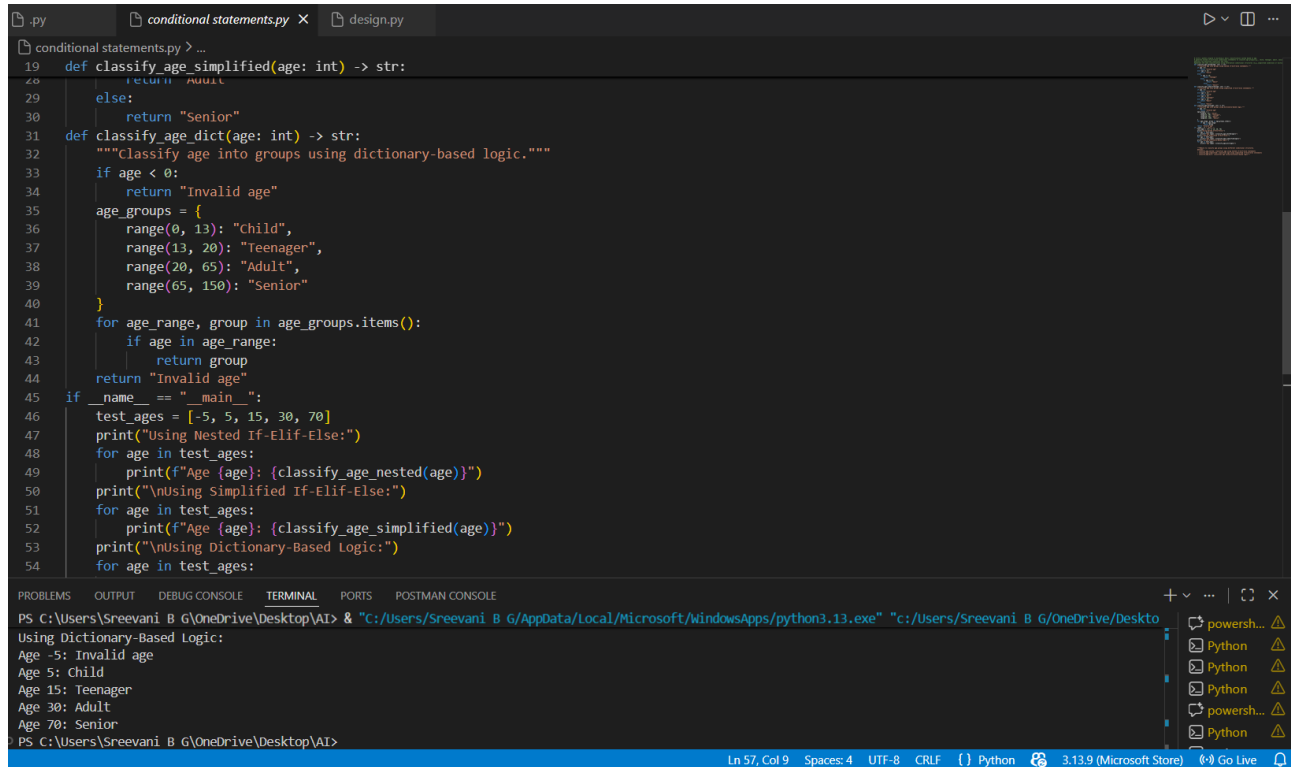
Task: Use AI tool to generate nested if-elif-else conditional statements to classify age groups (e.g., child, teenager, adult, senior).

Prompt: Write and analyze Python code to classify age groups using if-elif-else and an alternative conditional approach.

Expected Output:

- A Python function that classifies age into appropriate groups.
- Clear and correct conditional logic.
- Explanation of how the conditions work.

Output Screenshot:

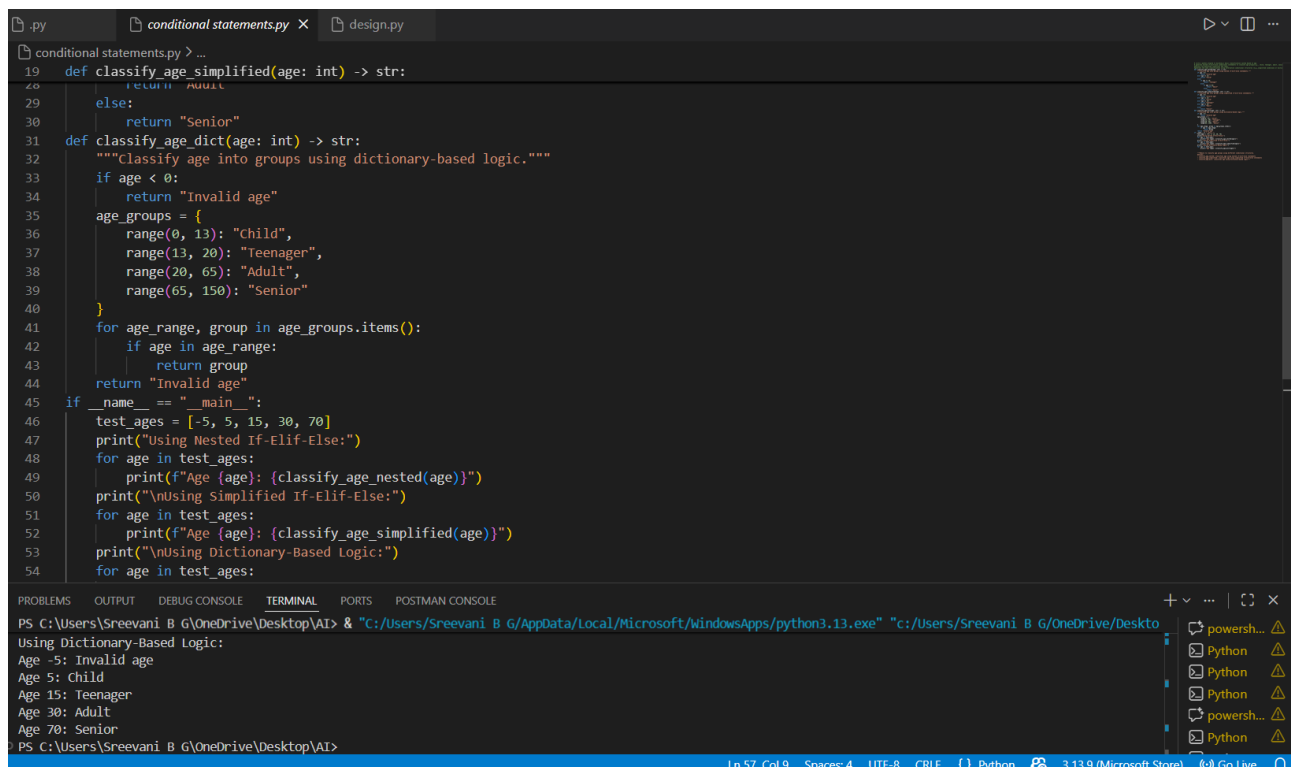


The screenshot shows a VS Code editor with a file named `conditional statements.py`. The code defines two functions: `classify_age_simplified` and `classify_age_dict`. The `classify_age_simplified` function uses nested if-elif-else statements to classify age groups. The `classify_age_dict` function uses a dictionary-based logic to classify age groups. The code also includes a main block that tests both functions with a list of ages: `[-5, 5, 15, 30, 70]`.

```
19 def classify_age_simplified(age: int) -> str:
20     return "Adult"
21 else:
22     return "Senior"
23 def classify_age_dict(age: int) -> str:
24     """Classify age into groups using dictionary-based logic."""
25     if age < 0:
26         return "Invalid age"
27     age_groups = {
28         range(0, 13): "Child",
29         range(13, 20): "Teenager",
30         range(20, 65): "Adult",
31         range(65, 150): "Senior"
32     }
33     for age_range, group in age_groups.items():
34         if age in age_range:
35             return group
36     return "Invalid age"
37 if __name__ == "__main__":
38     test_ages = [-5, 5, 15, 30, 70]
39     print("Using Nested If-Elif-Else:")
40     for age in test_ages:
41         print(f"Age {age}: {classify_age_simplified(age)}")
42     print("\nUsing Simplified If-Elif-Else:")
43     for age in test_ages:
44         print(f"Age {age}: {classify_age_simplified(age)}")
45     print("\nUsing Dictionary-Based Logic:")
46     for age in test_ages:
```

The terminal output shows the results of running the script:

```
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI> "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/Desktop/conditional statements.py"
Using Dictionary-Based Logic:
Age -5: Invalid age
Age 5: Child
Age 15: Teenager
Age 30: Adult
Age 70: Senior
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI>
```



The screenshot shows a VS Code editor with a file named `conditional statements.py`. The code defines two functions: `classify_age_simplified` and `classify_age_dict`. The `classify_age_simplified` function uses nested if-elif-else statements to classify age groups. The `classify_age_dict` function uses a dictionary-based logic to classify age groups. The code also includes a main block that tests both functions with a list of ages: `[-5, 5, 15, 30, 70]`.

```
19 def classify_age_simplified(age: int) -> str:
20     return "Adult"
21 else:
22     return "Senior"
23 def classify_age_dict(age: int) -> str:
24     """Classify age into groups using dictionary-based logic."""
25     if age < 0:
26         return "Invalid age"
27     age_groups = {
28         range(0, 13): "Child",
29         range(13, 20): "Teenager",
30         range(20, 65): "Adult",
31         range(65, 150): "Senior"
32     }
33     for age_range, group in age_groups.items():
34         if age in age_range:
35             return group
36     return "Invalid age"
37 if __name__ == "__main__":
38     test_ages = [-5, 5, 15, 30, 70]
39     print("Using Nested If-Elif-Else:")
40     for age in test_ages:
41         print(f"Age {age}: {classify_age_simplified(age)}")
42     print("\nUsing Simplified If-Elif-Else:")
43     for age in test_ages:
44         print(f"Age {age}: {classify_age_simplified(age)}")
45     print("\nUsing Dictionary-Based Logic:")
46     for age in test_ages:
```

The terminal output shows the results of running the script:

```
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI> "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/Desktop/conditional statements.py"
Using Dictionary-Based Logic:
Age -5: Invalid age
Age 5: Child
Age 15: Teenager
Age 30: Adult
Age 70: Senior
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI>
```

Analysis of Conditional Logic (if-elif-else):

- The conditions are checked one by one from top to bottom, and only the first matching block is executed.
- Invalid ages, such as negative values, are handled at the beginning to avoid incorrect classification.
- Each `elif` condition clearly defines an age range, ensuring there is no overlap between categories.
- The structure is simple and easy to understand, especially for beginners.
- This approach works well when the classification rules are straightforward and follow a clear sequence.

Explanation of the Alternative Logic:

- Age ranges are stored as `range` objects mapped to their respective age groups in a dictionary.
- The function iterates through the dictionary and checks which range the age belongs to.
- This separates **data (age ranges)** from **logic**, making the code more modular.
- Easy to modify or extend by just updating the dictionary.
- Useful when classification rules may change or grow over time.

Task Description #4: For and While Loops (Sum of First n Numbers)

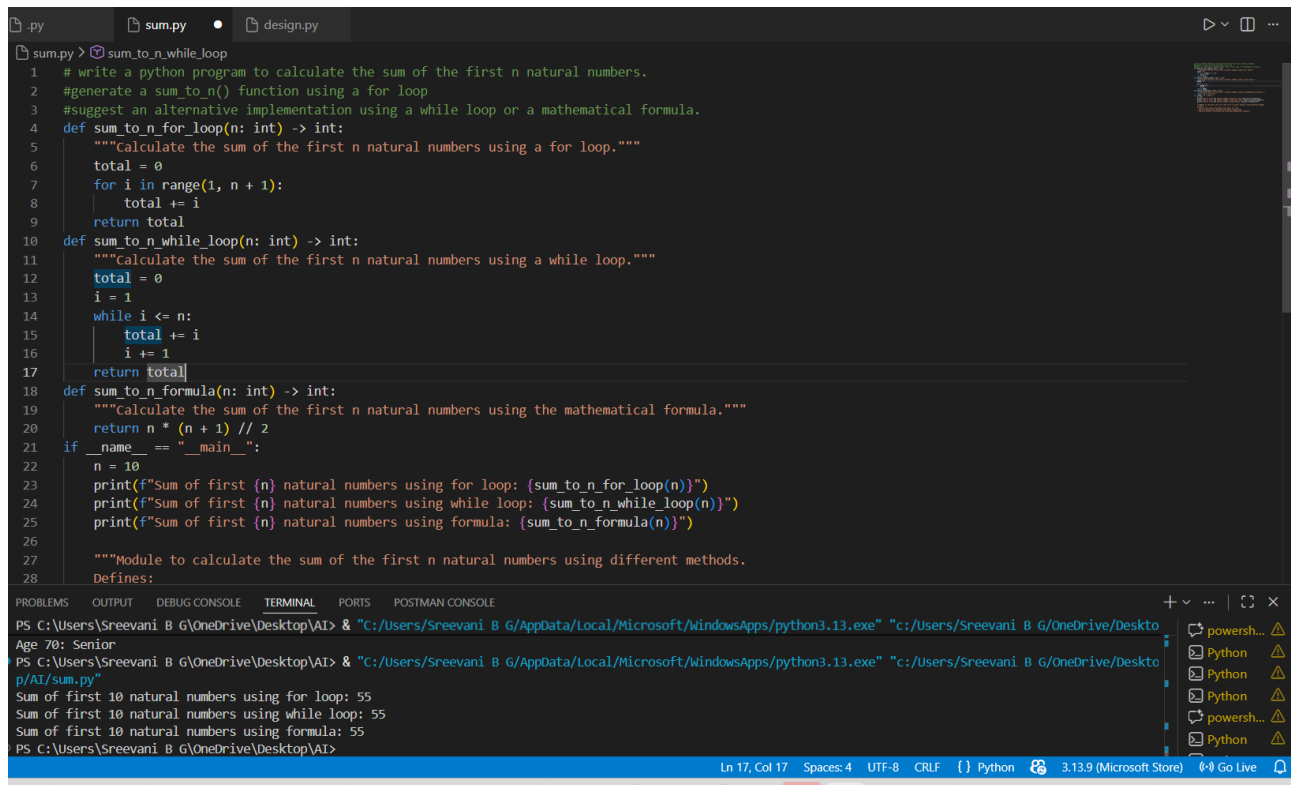
Task: Use AI assistance to generate a `sum_to_n()` function using a for loop.

Prompt: " write a python code to find the sum of first n natural numbers using for, while, and formula-based approaches."

Expected Output #4

- Python function to compute the sum of first n numbers.
- Correct output for sample inputs.
- Explanation and comparison of different approaches..

Output Screenshot:



```
sum.py> sum_to_n_while_loop
1 # write a python program to calculate the sum of the first n natural numbers.
2 #generate a sum_to_n() function using a for loop
3 #suggest an alternative implementation using a while loop or a mathematical formula.
4 def sum_to_n_for_loop(n: int) -> int:
5     """Calculate the sum of the first n natural numbers using a for loop."""
6     total = 0
7     for i in range(1, n + 1):
8         total += i
9     return total
10 def sum_to_n_while_loop(n: int) -> int:
11     """Calculate the sum of the first n natural numbers using a while loop."""
12     total = 0
13     i = 1
14     while i <= n:
15         total += i
16         i += 1
17     return total
18 def sum_to_n_formula(n: int) -> int:
19     """Calculate the sum of the first n natural numbers using the mathematical formula."""
20     return n * (n + 1) // 2
21 if __name__ == "__main__":
22     n = 10
23     print(f"Sum of first {n} natural numbers using for loop: {sum_to_n_for_loop(n)}")
24     print(f"Sum of first {n} natural numbers using while loop: {sum_to_n_while_loop(n)}")
25     print(f"Sum of first {n} natural numbers using formula: {sum_to_n_formula(n)}")
26
27     """Module to calculate the sum of the first n natural numbers using different methods.
28     Defines:
```

PS C:\Users\Sreevani B G\OneDrive\Desktop\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/Desktop/AI/sum.py"

Sum of first 10 natural numbers using for loop: 55
Sum of first 10 natural numbers using while loop: 55
Sum of first 10 natural numbers using formula: 55
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI>

Analysis of the for Loop Code

- The loop runs from 1 to n, adding each number to the variable `total`.
- Initialization of `total` to 0 ensures correct accumulation.
- The logic is simple, clear, and easy to understand.
- This approach works well when the number of iterations is known.

Task Description #5: Classes (Bank Account Class)

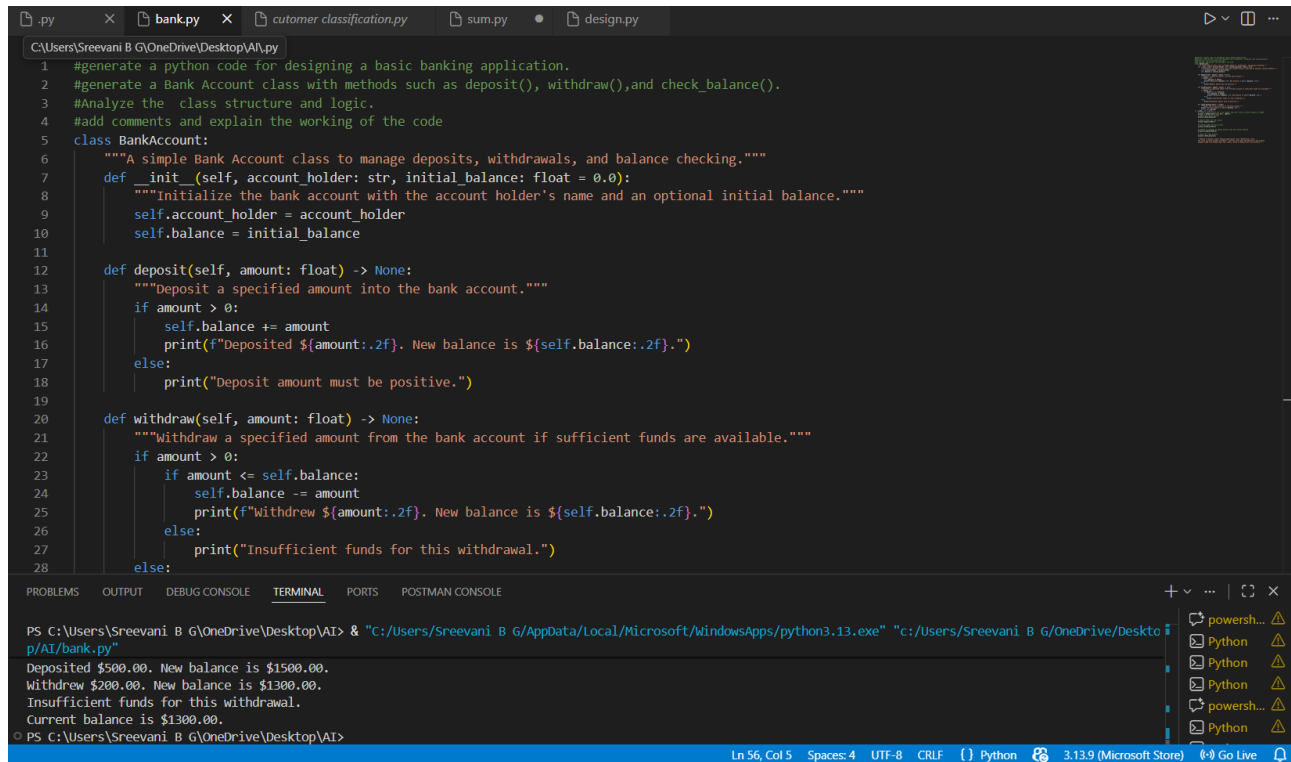
Task: Use AI tools to generate a Bank Account class with methods such as `deposit()`, `withdraw()`, and `check_balance()`.

Prompt: " write a Python code for Bank Account class with `deposit`, `withdraw`, and `balance check` methods."

Expected Output :

- Complete Python Bank Account class.
- Demonstration of deposit and withdrawal operations with updated balance.
- Well-commented code with a clear explanation..

Output Screenshot:



```
1 #generate a python code for designing a basic banking application.
2 #generate a Bank Account class with methods such as deposit(), withdraw(),and check_balance().
3 #Analyze the class structure and logic.
4 #add comments and explain the working of the code
5 class BankAccount:
6     """A simple Bank Account class to manage deposits, withdrawals, and balance checking."""
7     def __init__(self, account_holder: str, initial_balance: float = 0.0):
8         """Initialize the bank account with the account holder's name and an optional initial balance."""
9         self.account_holder = account_holder
10        self.balance = initial_balance
11
12    def deposit(self, amount: float) -> None:
13        """Deposit a specified amount into the bank account."""
14        if amount > 0:
15            self.balance += amount
16            print(f"Deposited ${amount:.2f}. New balance is ${self.balance:.2f}.")
17        else:
18            print("Deposit amount must be positive.")
19
20    def withdraw(self, amount: float) -> None:
21        """Withdraw a specified amount from the bank account if sufficient funds are available."""
22        if amount > 0:
23            if amount <= self.balance:
24                self.balance -= amount
25                print(f"Withdrew ${amount:.2f}. New balance is ${self.balance:.2f}.")
26            else:
27                print("Insufficient funds for this withdrawal.")
28        else:
29            print("Withdrawal amount must be positive.")
30
31    def check_balance(self) -> float:
32        """Return the current balance of the bank account."""
33        print(f"Current balance is ${self.balance:.2f}.")
34        return self.balance
35
36 if __name__ == "__main__":
37     # Create a bank account for a user named "John Doe" with an initial balance of $1000
38     account = BankAccount("John Doe", 1000.0)
39     # Check the initial balance
40     account.check_balance()
41
42     # Deposit $500 into the account
43     account.deposit(500.0)
44
45     # Withdraw $200 from the account
46     account.withdraw(200.0)
47
48     # Attempt to withdraw an amount greater than the current balance
49     account.withdraw(2000.0)
50
51     # Check the final balance
52     account.check_balance()
53
54     """Module to define a basic banking application with a BankAccount class.
55     The class includes methods to deposit, withdraw, and check the balance of the account.
```

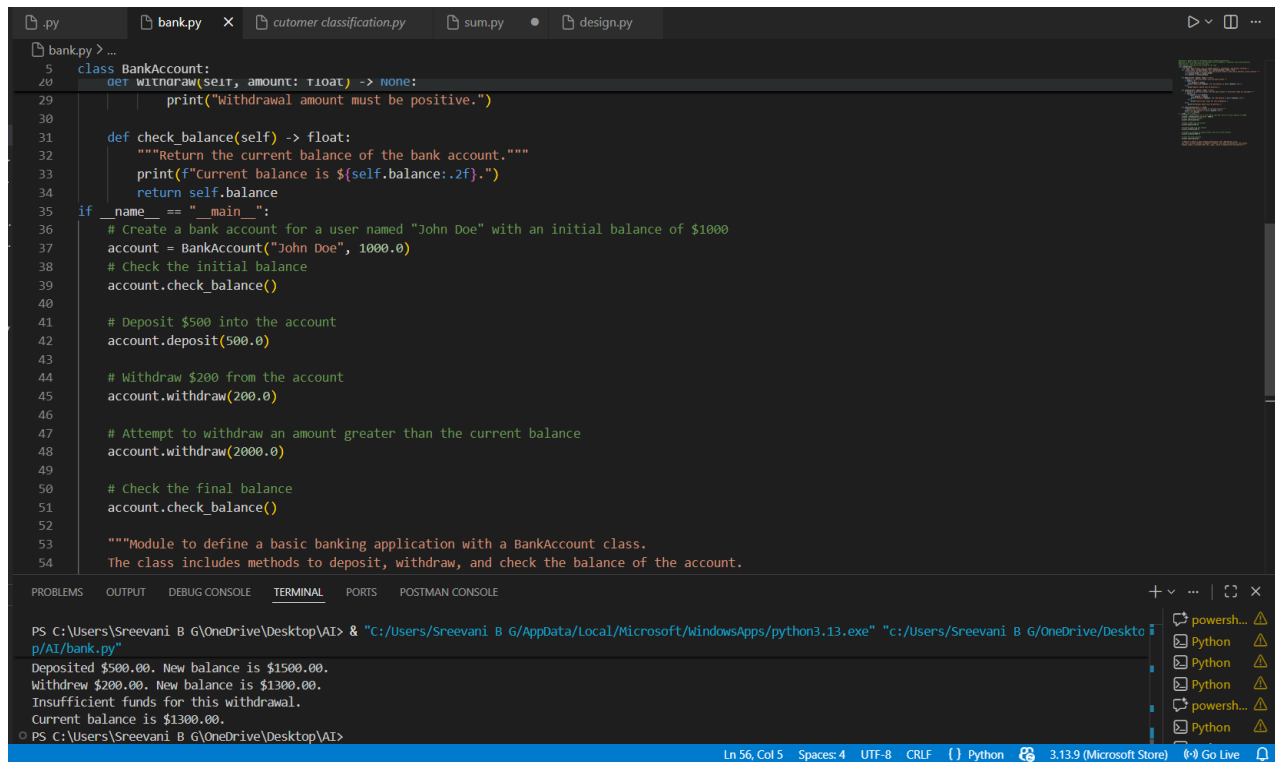
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

PS C:\Users\Sreevani B G\OneDrive\Desktop\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/Desktop/p/ai/bank.py"

Deposited \$500.00. New balance is \$1500.00.
Withdrew \$200.00. New balance is \$1300.00.
Insufficient funds for this withdrawal.
Current balance is \$1300.00.

PS C:\Users\Sreevani B G\OneDrive\Desktop\AI>

Ln 56, Col 5 Spaces: 4 UTF-8 CRLF Python 3.13.9 (Microsoft Store) Go Live



```
5 class BankAccount:
29     def withdraw(self, amount: float) -> None:
30         print("Withdrawal amount must be positive.")
31
32     def check_balance(self) -> float:
33         """Return the current balance of the bank account."""
34         print(f"Current balance is ${self.balance:.2f}.")
35         return self.balance
36
37 if __name__ == "__main__":
38     # Create a bank account for a user named "John Doe" with an initial balance of $1000
39     account = BankAccount("John Doe", 1000.0)
40     # Check the initial balance
41     account.check_balance()
42
43     # Deposit $500 into the account
44     account.deposit(500.0)
45
46     # Withdraw $200 from the account
47     account.withdraw(200.0)
48
49     # Attempt to withdraw an amount greater than the current balance
50     account.withdraw(2000.0)
51
52     # Check the final balance
53     account.check_balance()
54
55     """Module to define a basic banking application with a BankAccount class.
56     The class includes methods to deposit, withdraw, and check the balance of the account.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

PS C:\Users\Sreevani B G\OneDrive\Desktop\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/Desktop/p/ai/bank.py"

Deposited \$500.00. New balance is \$1500.00.
Withdrew \$200.00. New balance is \$1300.00.
Insufficient funds for this withdrawal.
Current balance is \$1300.00.

PS C:\Users\Sreevani B G\OneDrive\Desktop\AI>

Ln 56, Col 5 Spaces: 4 UTF-8 CRLF Python 3.13.9 (Microsoft Store) Go Live

Analysis of Class Structure and Logic:

- The BankAccount class represents a real-world bank account.
- The constructor (`__init__`) initializes account details and balance.
- The `deposit()` method ensures only positive amounts are added.
- The `withdraw()` method checks for valid amount and sufficient balance.
- The `check_balance()` method displays the current balance.
- Input validation prevents invalid transactions.

Explanation of Code Working:

- A BankAccount object is created with an initial balance to represent a real bank account.
- When a deposit is made, the amount is first validated and then added to the existing balance.
- During withdrawal, the code checks whether enough balance is available before deducting the amount.
- After each transaction, the updated balance is displayed to show the current account status.
- Overall, the code is easy to read, well-structured, and follows good programming practices.